

PROJECT REPORT

Intelligent Traffic Monitoring and Vehicle Tracking System

A Computer-Vision Based System for Automated Vehicle Detection, Tracking, and Traffic Density Estimation

Student Name	Swagata Ojha
Registration No.	24BAI10921
Course	Computer Vision
Submission Date	18 Sep. 26

Table of Contents

- 1. Introduction**
- 2. Problem Statement**
- 3. Functional Requirements**
- 4. Non-Functional Requirements**
- 5. System Architecture**
- 6. Design Diagrams**
- 7. Design Decisions & Rationale**
- 8. Implementation Details**
- 9. Screenshots / Results**
- 10. Testing Approach**
- 11. Challenges Faced**
- 12. Learnings & Key Takeaways**
- 13. Future Enhancements**
- 14. References**

1. Introduction

Traffic monitoring is a foundational capability for modern transportation management, congestion analysis, and urban planning. Historically, this has depended on manual observation or purpose-built hardware such as inductive loop sensors and radar units — approaches that are costly to install and difficult to scale across a city's road network.

This project, the Intelligent Traffic Monitoring and Vehicle Tracking System, explores an alternative: using ordinary traffic video (from a fixed CCTV camera, a traffic pole, or a webcam) and classical computer-vision techniques to automatically detect vehicles, track their movement across frames, and estimate live traffic density — without requiring specialized sensors, a GPU, or a large pretrained deep-learning model.

The system is built entirely in Python using OpenCV, and follows a modular pipeline architecture: preprocessing, background subtraction, detection, tracking, and traffic analysis are each implemented as independent, testable components, with all thresholds centralized in a single configuration file. This design keeps the system transparent, easy to reason about, and straightforward to extend — for example, by substituting the contour-based detector with a trained deep-learning model without touching the rest of the pipeline.

This report documents the problem the system addresses, its requirements, architecture, design decisions, implementation, testing approach, and the challenges encountered while building it, along with reflections on what was learned and how the system could be extended further.

2. Problem Statement

Traditional traffic monitoring systems rely heavily on manual observation or expensive dedicated infrastructure, making continuous, city-scale monitoring difficult and inefficient. A large amount of traffic camera footage is recorded but never actively analyzed, leaving valuable visual data untapped for traffic management purposes.

There is a need for an automated computer-vision system that can process ordinary traffic video, detect vehicles, track their movement across frames, and extract useful information such as vehicle count, movement patterns, and traffic density — using techniques lightweight enough to run without specialized hardware, so the resulting system remains transparent, reproducible, and accessible for small-scale deployments (a single intersection, a parking-lot entrance, a campus road) where dedicated sensing hardware is not cost-justified.

The system aims to process video footage through image preprocessing, background subtraction, object detection, feature extraction, and motion analysis to identify vehicles and monitor their movement, using the extracted information to estimate traffic density and support intelligent traffic management decisions.

3. Functional Requirements

The system shall provide the following capabilities:

- Accept a traffic video file (e.g., .mp4, .avi) or a live webcam feed as input.
- Preprocess each frame by resizing, enhancing contrast (CLAHE), and reducing noise (Gaussian blur).
- Model the video's background and extract a foreground mask isolating moving objects (MOG2 background subtraction with morphological cleanup).
- Detect vehicle candidates from the foreground mask using contour analysis, filtered by a configurable minimum/maximum area and aspect ratio.
- Classify each detected vehicle into a type (car, bike, bus, truck) using a size-based heuristic.
- Assign each detected vehicle a persistent, unique ID and track it across consecutive frames using centroid-distance matching.
- Tolerate brief occlusions or missed detections for a configurable number of frames before dropping a track.
- Record each tracked vehicle's trajectory (recent centroid history) and cumulative displacement.
- Compute the live (on-screen) vehicle count for every processed frame.
- Maintain a cumulative count of unique vehicles seen, broken down by vehicle type.
- Classify current traffic density into LOW / MEDIUM / HIGH based on configurable vehicle-count thresholds.
- Render bounding boxes, vehicle IDs, and motion trails onto each output frame.
- Overlay a live statistics panel (vehicle count, density, per-type counts) on the output video.
- Write the fully annotated video to disk in a user-specified location.
- Generate a vehicle-count-over-time line graph (PNG) once processing completes.
- Export a final summary of statistics (total unique vehicles, counts by type, frames processed) as JSON.
- Allow every detection, tracking, and density threshold to be tuned from a single configuration file without editing pipeline logic.
- Provide an automated test suite that verifies detection, classification, and tracking logic using synthetically generated data (no real traffic video required to test).

4. Non-Functional Requirements

4.1 Performance

- The pipeline should process typical traffic video (resized to a working width of 960px) at a rate suitable for near-real-time review on standard consumer CPU hardware, without requiring a GPU.

4.2 Reliability

- The tracker should tolerate brief, intermittent detection failures (e.g., partial occlusion) without discarding a vehicle's identity, up to a configurable number of frames.
- The system should degrade gracefully on video sources with variable lighting or minor camera jitter, rather than crashing.

4.3 Usability

- All operational parameters (thresholds, codecs, output paths) should be exposed through a single, well-documented configuration file rather than scattered throughout the code.
- Command-line usage should require no more than a video path to produce a complete, annotated output.

4.4 Maintainability

- The codebase should be organized into small, single-responsibility modules (preprocessing, detection, tracking, analysis, visualization) that can be independently tested, replaced, or extended.
- The detection module should be swappable (e.g., for a trained deep-learning detector) without requiring changes to the tracking or analysis modules.

4.5 Portability

- The system should run cross-platform (Windows, macOS, Linux) using only pip-installable dependencies (OpenCV, NumPy, SciPy, Matplotlib), with no GPU or large model download required.

4.6 Testability

- Core logic (detection filtering, size-based classification, tracker ID assignment, density labeling) should be verifiable via automated unit tests using synthetically generated inputs, independent of any specific real-world video.

5. System Architecture

The system follows a layered pipeline architecture. A configuration layer centralizes every tunable parameter; a processing pipeline applies a fixed sequence of computer-vision stages to each frame; and a presentation layer renders the results into an annotated video, a summary graph, and a JSON statistics file.

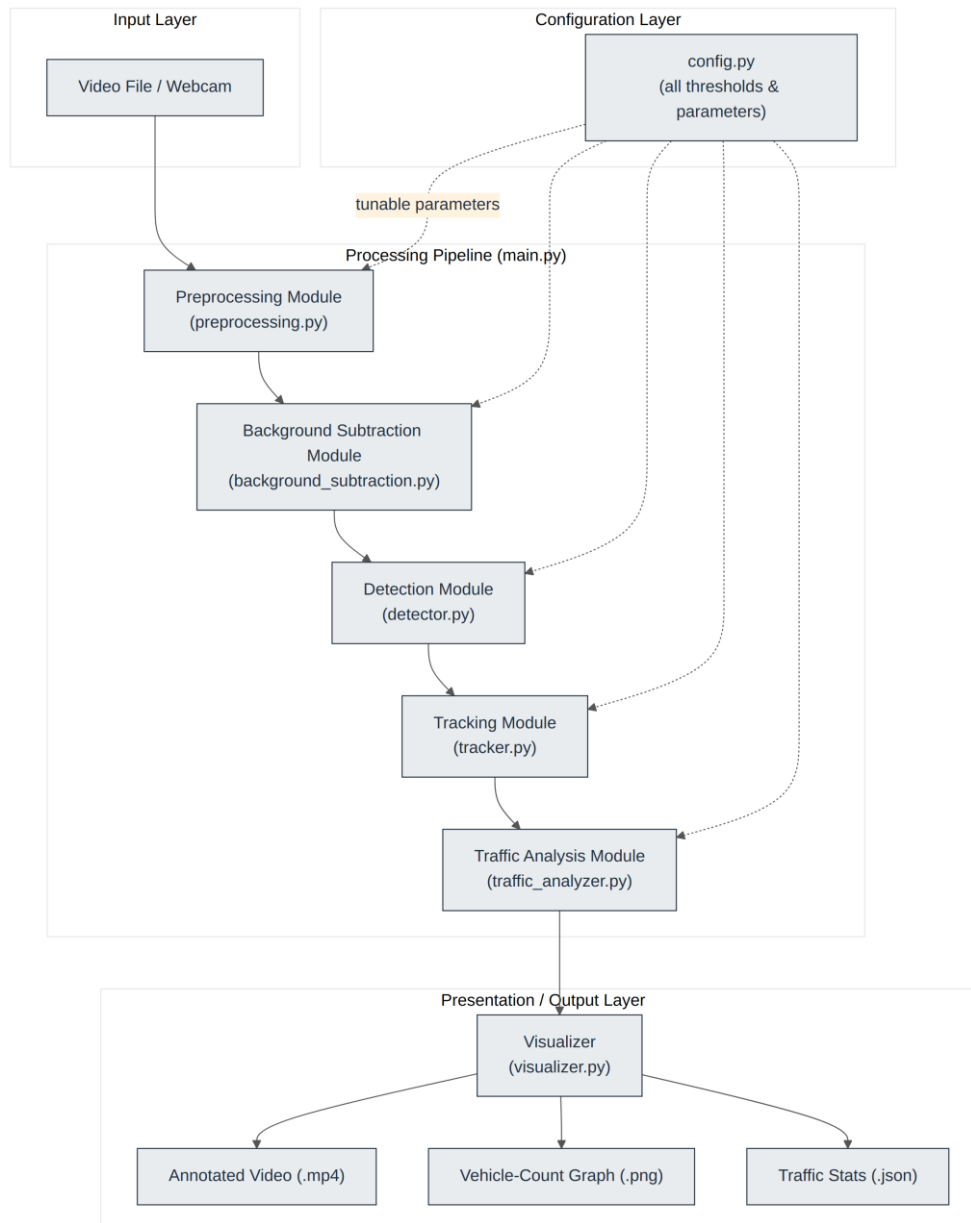


Figure 5.1 — Layered system architecture

Each processing-pipeline module (preprocessing, background subtraction, detection, tracking, traffic analysis) is implemented as an independent Python module with a narrow, well-defined interface, so any stage can be tested, replaced, or extended in isolation. `main.py` acts purely as an orchestrator: it wires the modules together frame by frame and does not itself contain any detection or tracking logic.

6. Design Diagrams

6.1 Use Case Diagram

Two actors interact with the system: the Traffic Operator / Analyst, who supplies video, configures thresholds, and reviews the output; and the Course Instructor / Evaluator, who runs the automated test suite and reviews results for assessment purposes.

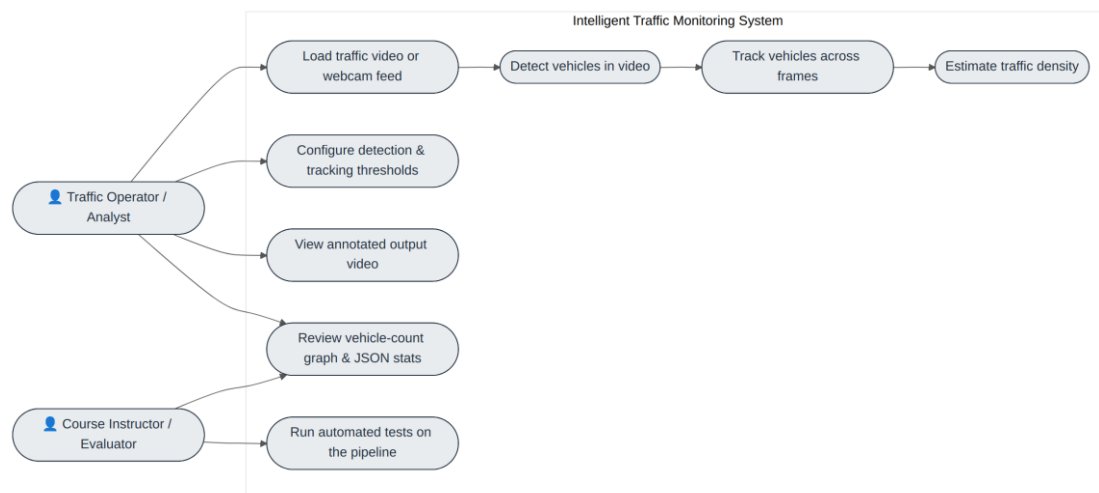


Figure 6.1 — Use case diagram

6.2 Workflow Diagram

The workflow diagram traces a single frame's journey through the full pipeline, from raw input to the three output artifacts. This loop repeats for every frame in the input video.

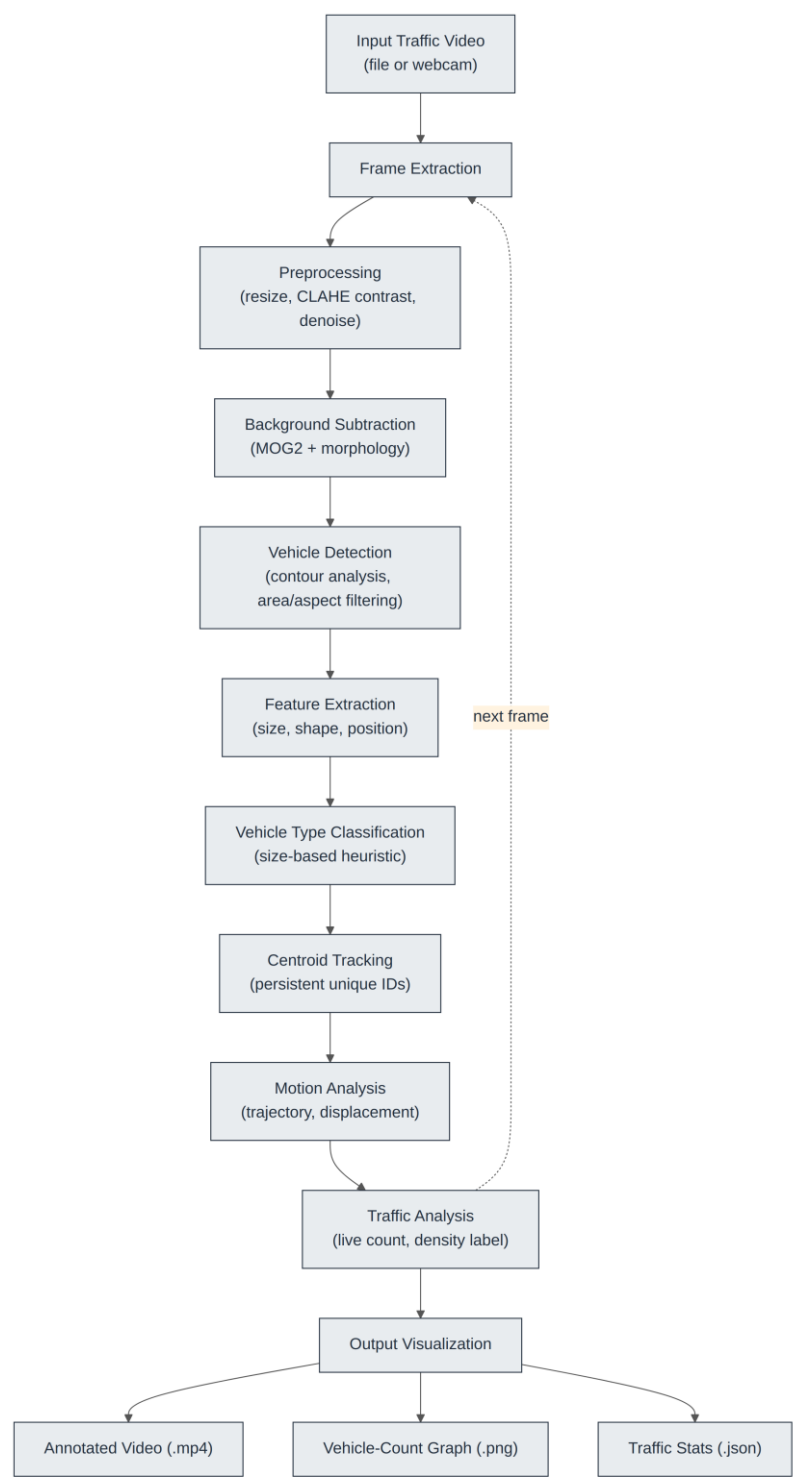


Figure 6.2 — End-to-end processing workflow

6.3 Sequence Diagram

The sequence diagram shows the interaction between `main.py` (the orchestrator) and each pipeline module for a single processed frame, illustrating the request/response flow between components.

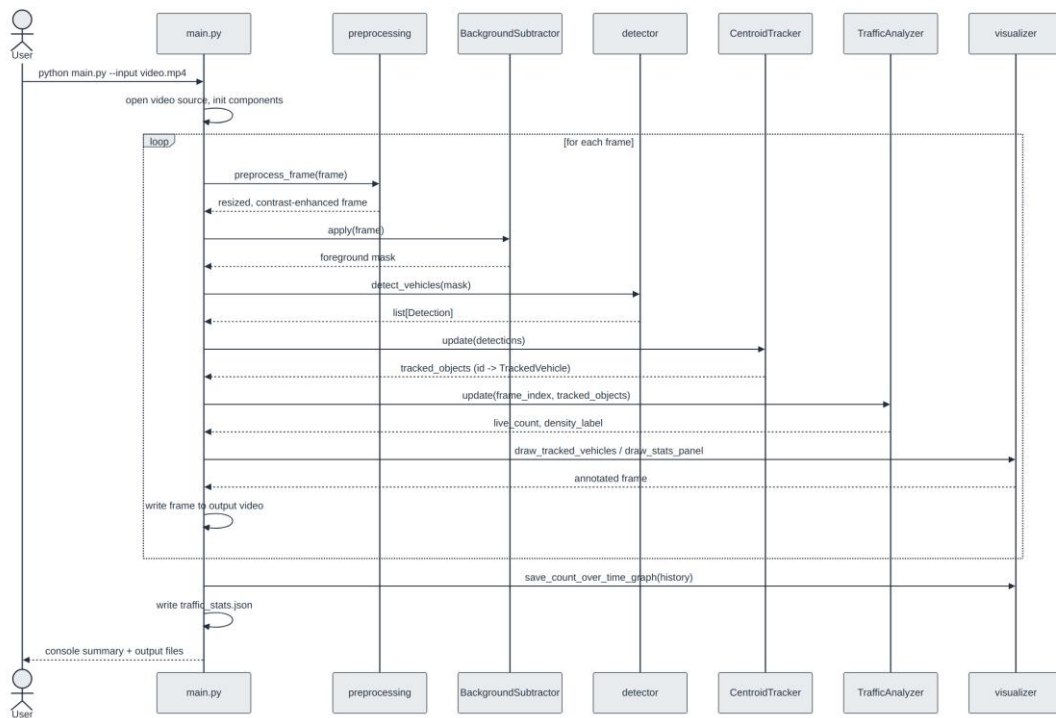


Figure 6.3 — Per-frame processing sequence

6.4 Class / Component Diagram

The class diagram shows the key data classes (Detection, TrackedVehicle) and the stateful components that operate on them (CentroidTracker, TrafficAnalyzer, BackgroundSubtractor), along with the module-level components (Visualizer, MainPipeline) that coordinate them.

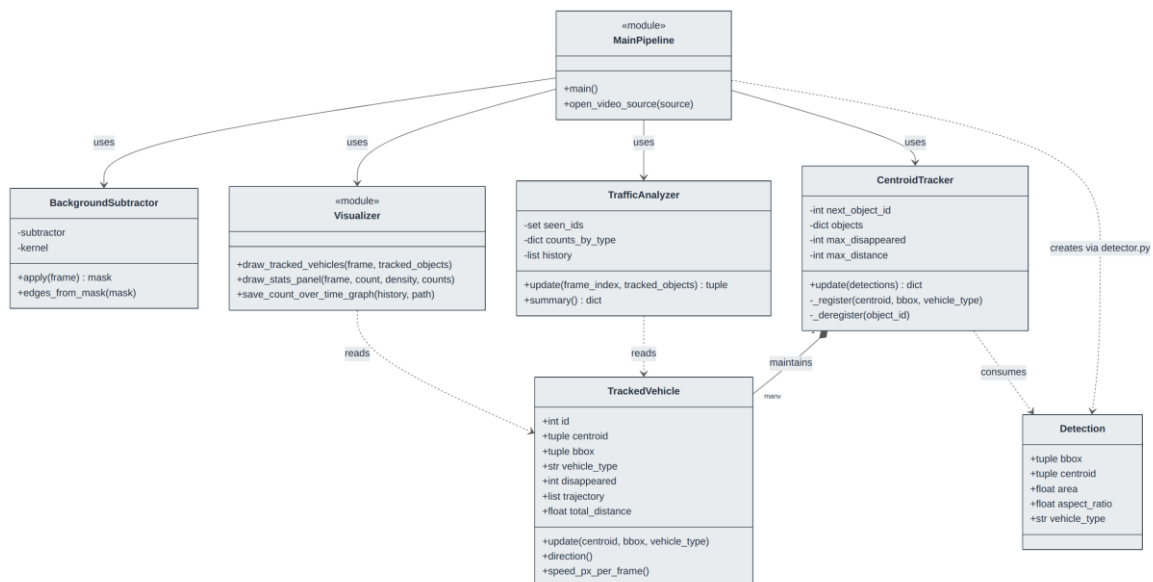


Figure 6.4 — Class / component diagram

6.5 ER Diagram

Not applicable. This system does not use a database or any other persistent-storage engine — it reads a video file (or webcam stream) as input and writes its results directly to the filesystem as an

annotated video (.mp4), a chart image (.png), and a statistics summary (.json). There is no entity-relationship schema to model, since no relational or document data store is part of the design.

7. Design Decisions & Rationale

7.1 Classical computer vision over deep learning

The detector uses background subtraction (MOG2) and contour analysis rather than a trained deep-learning model such as YOLO. This keeps the system dependency-light, fully reproducible offline (no model weights to download), and runnable on ordinary CPU hardware — appropriate for an academic prototype and small-scale deployments. The detector module is deliberately isolated behind a narrow interface so a deep-learning detector could be substituted later without changing the tracker or analyzer.

7.2 Centroid-distance tracking over more complex trackers

A greedy centroid-distance tracker (nearest-neighbor matching between existing tracks and new detections, using SciPy's `cdist`) was chosen over more sophisticated approaches (e.g., Kalman-filter-based tracking or deep re-identification embeddings). For the fixed-camera, moderate-frame-rate scenario this system targets, centroid distance is a cheap, easy-to-reason-about heuristic that performs well, and it keeps the codebase approachable for an academic setting while still supporting persistent IDs and occlusion tolerance via a 'disappeared frame' counter.

7.3 Size-based vehicle-type classification

Rather than training a classifier, vehicle type (car / bike / bus / truck) is inferred from the detected blob's pixel area against configurable thresholds. This is intentionally simple — it requires no labeled training data — but is clearly documented as a heuristic, with the codebase structured so a trained classifier could replace it without affecting detection or tracking.

7.4 Centralized configuration

Every threshold used across preprocessing, background subtraction, detection, tracking, and density estimation lives in a single `src/config.py` file, rather than being scattered as magic numbers throughout the pipeline modules. This makes the system easy to tune for a new camera angle or lighting condition without touching processing logic, and makes the effect of each parameter easy to document in one place.

7.5 Morphological cleanup of the foreground mask

Raw MOG2 output is noisy (isolated speckles, small holes inside vehicle blobs). A morphological opening (removes small noise) followed by a closing (fills small holes) is applied before contour detection, substantially improving the quality of detected blobs at a low computational cost compared to more elaborate denoising.

7.6 Modular, single-responsibility pipeline stages

Each pipeline stage (preprocessing, background subtraction, detection, tracking, traffic analysis, visualization) is a separate module with a narrow interface. This was a deliberate trade-off favoring testability and maintainability: each module can be unit-tested with synthetic inputs independent of

a real video, and any single stage can be replaced (e.g., a different background model, or a trained detector) without cascading changes elsewhere.

7.7 Synthetic-video testing strategy

Rather than requiring a real, licensed traffic-video dataset to validate the pipeline, a synthetic-video generator (`tests/generate_sample_video.py`) creates a short clip with moving rectangles of known size and speed. This lets both automated tests and a manual end-to-end smoke test run anywhere, deterministically, without external data dependencies — at the cost of not validating real-world detection accuracy, which must ultimately be checked against genuine footage.

8. Implementation Details

The system is implemented in Python 3, using OpenCV for video I/O, image processing, and drawing; NumPy and SciPy for numerical operations (including the tracker's distance-matrix computation); and Matplotlib for generating the summary graph.

8.1 Module responsibilities

Module	Responsibility
config.py	Centralizes all tunable thresholds and parameters.
preprocessing.py	Resizes frames, applies CLAHE contrast enhancement, and denoises via Gaussian blur.
background_subtraction.py	Wraps OpenCV's MOG2 subtractor; applies shadow removal and morphological open/close to produce a clean foreground mask.
detector.py	Finds contours in the foreground mask, filters by area/aspect ratio, and classifies vehicle type by size.
tracker.py	Maintains a dictionary of TrackedVehicle objects; matches new detections to existing tracks by minimum centroid distance; ages out and drops tracks that disappear for too long.
traffic_analyzer.py	Aggregates tracker output into a live count, cumulative per-type counts, and a LOW/MEDIUM/HIGH density label.
visualizer.py	Draws bounding boxes, IDs, and trails onto frames; renders the live stats panel; saves the final count-over-time graph.
main.py	Orchestrates the per-frame pipeline, handles CLI arguments, and writes all output artifacts.

8.2 Key algorithms

Background subtraction and mask cleanup

Frames are passed through `cv2.createBackgroundSubtractorMOG2`, shadow pixels (marked gray, value 127) are thresholded out, and the result is cleaned with a morphological opening (2 iterations) followed by a closing (2 iterations) using an elliptical structuring element.

Vehicle detection and classification

`cv2.findContours` locates blobs in the cleaned mask. Each contour is filtered by area (`config.MIN_CONTOUR_AREA – MAX_CONTOUR_AREA`) and bounding-box aspect ratio (`config.MIN_ASPECT_RATIO – MAX_ASPECT_RATIO`) to reject noise and shadow artifacts. Surviving blobs are classified by area against `config.VEHICLE_TYPE_THRESHOLDS` to produce a vehicle-type label.

Centroid tracking

For each frame, a distance matrix (SciPy's `cdist`) is computed between existing tracked centroids and new detection centroids. Matches are assigned greedily in order of increasing distance, subject to a maximum-distance cutoff (`config.MAX_MATCH_DISTANCE`). Unmatched existing tracks have their 'disappeared' counter incremented and are dropped once it exceeds `config.MAX_DISAPPEARED_FRAMES`; unmatched detections register as new tracks with fresh IDs.

Traffic density estimation

The live on-screen vehicle count is compared against two configurable cutoffs (`config.DENSITY_THRESHOLDS`) to produce a LOW / MEDIUM / HIGH label, recorded every frame alongside the live count to build the count-over-time history used for the final graph.

8.3 Example: running the pipeline

```
$ python main.py --input sample_data/traffic.mp4 --output output/result.mp4

[INFO] Starting processing. Source FPS: 25.00, total frames reported: 150
[INFO] Processed 50 frames (37.9 fps) | live count: 4 | density: LOW
[INFO] Processed 100 frames (39.3 fps) | live count: 4 | density: LOW
[INFO] Processed 150 frames (39.6 fps) | live count: 3 | density: LOW

[DONE] Processing complete.
Frames processed      : 150
Total unique vehicles : 5
Counts by type       : {'bike': 4, 'car': 1}
Annotated video saved to : output/result.mp4
Graph saved to       : output/vehicle_count_over_time.png
Stats JSON saved to  : output/traffic_stats.json
```

9. Screenshots / Results

The pipeline was validated end-to-end using a short synthetically generated traffic clip (moving rectangles of known size and speed), produced by tests/generate_sample_video.py. This provides a deterministic way to confirm the full pipeline runs correctly without depending on a licensed real-world video dataset.

9.1 Annotated output frame

Each output frame shows bounding boxes and a persistent ID for every tracked vehicle, a short motion trail, and a live statistics panel (on-screen vehicle count, traffic density, and counts by type).

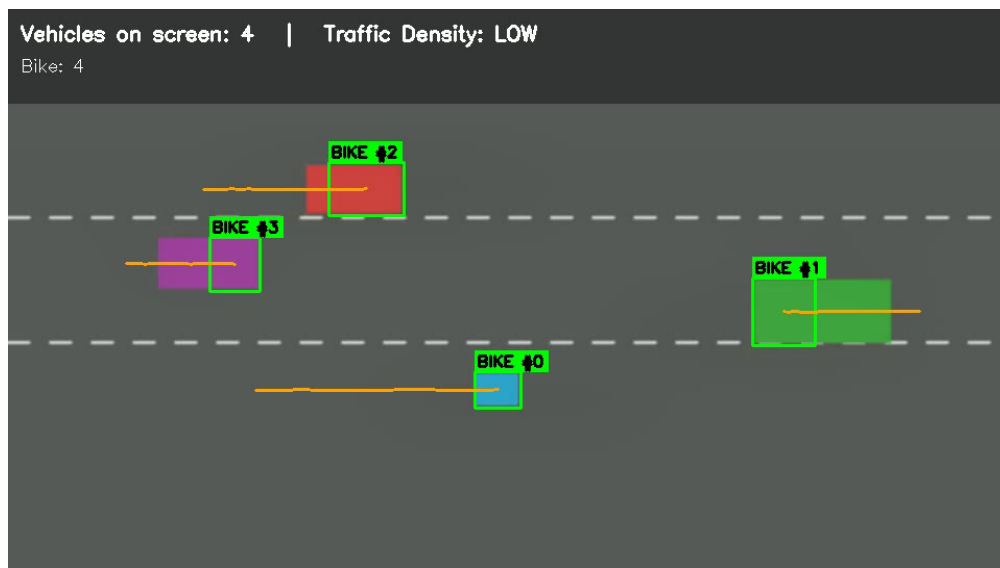


Figure 9.1 — Annotated output frame with tracked vehicles and live stats panel

9.2 Vehicle-count-over-time graph

At the end of a run, the system plots the on-screen vehicle count for every processed frame, giving a quick visual summary of how traffic volume varied over the clip.

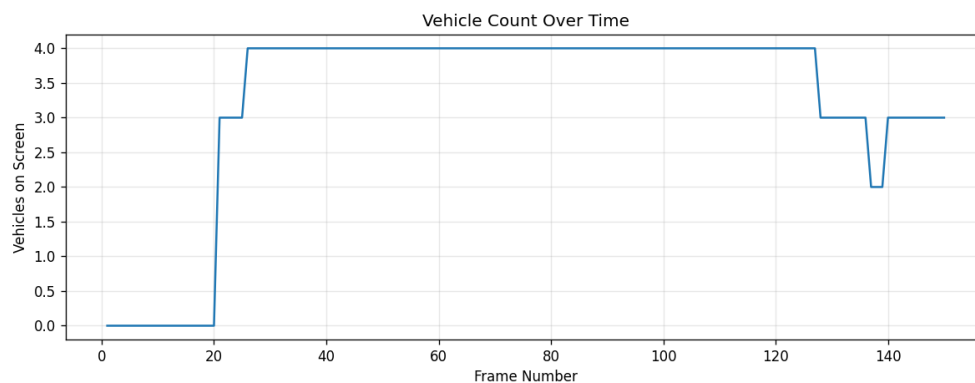


Figure 9.2 — Vehicle count over time for the test run

9.3 Final statistics summary (JSON)

```
{
  "total_unique_vehicles": 5,
  "counts_by_type": {
    "bike": 4,
    "car": 1
  },
  "frames_processed": 150,
  "annotated_video": "output/result.mp4",
  "count_over_time_graph": "output/vehicle_count_over_time.png"
}
```

10. Testing Approach

Testing combines automated unit tests against synthetically constructed inputs with an end-to-end manual smoke test, so the pipeline's correctness can be verified without depending on a real, licensed traffic-video dataset.

10.1 Automated unit tests (pytest)

tests/test_pipeline.py exercises the core decision logic directly:

- detect_vehicles() is tested against a synthetic binary mask with known rectangular blobs, verifying the expected number of detections are found.
- classify_by_size() is tested to confirm it returns one of the recognized vehicle-type labels for a given blob area.
- CentroidTracker is tested by feeding it detections from two successive synthetic masks — one where a blob has moved slightly — and verifying the same object ID is retained across frames.
- density_label() is tested against the LOW / MEDIUM / HIGH boundary values to confirm correct thresholding.

10.2 End-to-end smoke test

tests/generate_sample_video.py produces a short video containing rectangles of known size, moving at known speeds against a static background. Running main.py against this video and inspecting the annotated output, the console summary, and the generated graph/JSON confirms the full pipeline — preprocessing through output visualization — runs correctly end-to-end, without needing a real dataset.

10.3 Clean-environment verification

Before delivery, the project was verified in a fresh virtual environment using only the pinned versions in requirements.txt: a new venv was created, dependencies were installed exactly as documented in the README, the synthetic video was generated, main.py was run against it, and the automated test suite was executed — confirming the documented setup steps work exactly as written for a new user.

10.4 What is not covered by this test suite

Recognition accuracy on real, varied traffic footage (different lighting, camera angles, weather, and vehicle densities) is not exercised by the automated tests, since that depends on real video the tests cannot generate synthetically. This is a known limitation — see Section 11 (Challenges Faced) and Section 13 (Future Enhancements).

11. Challenges Faced

11.1 Validating the pipeline without a real dataset

Confirming that the full pipeline runs correctly end-to-end required test input, but obtaining and including a real, licensed traffic-video dataset was impractical for this project. This was addressed by building a synthetic-video generator that produces moving shapes of known size and speed — sufficient to prove the pipeline's mechanics work correctly, though it cannot substitute for validating detection accuracy on genuine footage.

11.2 Noisy background-subtraction output

Raw MOG2 foreground masks contained significant speckle noise and holes inside vehicle silhouettes, which produced fragmented or false contours. This was mitigated with a morphological opening-then-closing step, tuned by iterating on the kernel size and iteration count until vehicle blobs were solid and contour-friendly without merging genuinely separate vehicles.

11.3 Choosing tracking-distance and disappearance thresholds

Setting `MAX_MATCH_DISTANCE` and `MAX_DISAPPEARED_FRAMES` involved a trade-off: too strict, and tracks were dropped and re-assigned new IDs during brief occlusions or fast motion; too lenient, and distinct nearby vehicles risked being merged or mismatched. These were tuned empirically against the synthetic test video and documented in `config.py` so they can be re-tuned for a different camera setup.

11.4 Size-based vehicle classification is camera-distance-dependent

Because vehicle type is inferred from blob pixel area, the same physical vehicle produces very different blob sizes depending on how far it is from the camera and the camera's resolution/zoom. This is documented clearly as a heuristic limitation in both the code and this report, with `VEHICLE_TYPE_THRESHOLDS` exposed as a per-deployment tunable rather than a fixed constant.

11.5 Keeping the codebase swappable for a future deep-learning detector

Designing the detector module's interface (a plain list of Detection objects with `bbox`, `centroid`, `area`, and `vehicle_type`) so that a future deep-learning-based detector could be substituted without touching the tracker or analyzer required an extra round of interface design up front, rather than tightly coupling detection logic to the rest of the pipeline.

12. Learnings & Key Takeaways

- Classical computer-vision techniques (background subtraction, contour analysis, centroid tracking) remain effective for well-constrained scenarios like a fixed-camera traffic feed, and are far cheaper to deploy and reason about than a deep-learning pipeline when that constraint holds.
- A clean, narrow interface between pipeline stages (e.g., a simple Detection object) pays off quickly — it made unit testing straightforward and leaves a clear path to upgrade any single stage (e.g., swapping in a trained detector) later.
- Centralizing every threshold in one configuration file made the system dramatically easier to tune and to explain — both to a reader of the code and in this report — than having constants scattered across modules.
- Synthetic test data, while unable to validate real-world accuracy, is a powerful tool for verifying a pipeline's mechanical correctness (does it run end-to-end, do IDs persist correctly, does the graph get generated) in a fully reproducible, dependency-free way.
- Heuristic components (like size-based vehicle classification) are useful and valid for a prototype, but must be clearly documented as heuristics with known limitations, rather than presented as more accurate than they are.
- Morphological post-processing of a noisy binary mask is a small addition that meaningfully improves downstream detection quality, and is a good general-purpose technique for classical CV pipelines.

13. Future Enhancements

- Swap the contour-based detector for a trained deep-learning model (e.g., YOLO) for higher detection accuracy and more reliable vehicle-type classification, using the existing Detection interface so the tracker and analyzer require no changes.
- Add camera calibration to convert pixel displacement into real-world speed estimates.
- Add wrong-direction and lane-wise vehicle counting for richer traffic analytics.
- Add congestion/anomaly detection (e.g., stalled vehicles, sudden stops) built on top of the existing trajectory data already recorded per vehicle.
- Support multiple camera feeds and fuse their outputs into a single, city-level traffic dashboard.
- Automatically generate periodic traffic reports (e.g., hourly summaries) from the JSON statistics output.
- Replace the greedy centroid tracker with a more robust multi-object tracker (e.g., a Kalman-filter-based or deep re-identification approach) for busier scenes with more frequent occlusions.
- Validate detection and tracking accuracy quantitatively against a real, annotated traffic-video dataset, rather than relying solely on the synthetic smoke test.

14. References

- OpenCV Documentation — Background Subtraction.
https://docs.opencv.org/4.x/d1/dc5/tutorial_background_subtraction.html
- OpenCV Documentation — Contour Features.
https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html
- Zivkovic, Z. (2004). Improved Adaptive Gaussian Mixture Model for Background Subtraction. Proceedings of the 17th International Conference on Pattern Recognition (ICPR).
- SciPy Documentation — `scipy.spatial.distance.cdist`.
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cdist.html>
- Matplotlib Documentation. <https://matplotlib.org/stable/index.html>
- Rosebrock, A. Simple Object Tracking with OpenCV (centroid tracking algorithm reference). PyImageSearch.
- NumPy Documentation. <https://numpy.org/doc/>